Exp5 - weka
Exp6 - apriori

Exp3 - SQL + python

Exp4 - weka + python

Python

```python
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv('filepath')

print("Original Data:")
print(df)

print("\nFirst 10 Rows:")
print(df.head(10))

print("\nStatistical Description:")
print(df.describe())

print("\nMissing Values:")
print(df.isnull().sum())

df['Name'] = df['Name'].fillna('Unknown')
df['Marks'] = df['Marks'].fillna(df['Marks'].mean())

print("\nCleaned Data:")
print(df)

sns.histplot(df['Marks'], bins=5, kde=True)
plt.title('Histogram of Marks')
plt.xlabel('Marks')
plt.ylabel('Frequency')
plt.show()

plt.pie(
    df['Name'].value_counts(),
    autopct='%1.1f%%'
)

plt.title('Pie Chart of Students by Name')
plt.show()

sns.boxplot(x=df['Marks'])
plt.title('Boxplot of Marks')
```

```
plt.show()
```

Exp 7

Python

```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import classification_report, confusion_matrix


df = pd.read_csv('/content/sample_data/EEG Eye State.csv')

X = df.drop('eyeDetection', axis=1)
y=df['eyeDetection']

scalar = StandardScaler()
X_scaled = scalar.fit_transform(X)

kmeans = KMeans(n_clusters=2, random_state=42, n_init=1)
clusters = kmeans.fit_predict(X_scaled)

mapped_cluster = np.where(
    clusters==0,
    1 if np.mean(y[clusters==0]) >0.5 else 0,
    1 if np.mean(y[clusters==1]) >0.5 else 0
)

print(classification_report(y, mapped_cluster))
print(confusion_matrix(y, mapped_cluster))
```

Exp 8

```python
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, linkage

# Dataset
data = pd.read_csv("./hierarchical_data.csv")

# Select numeric columns
X = data[["Age", "Marks"]]

# Create dendrogram
linked = linkage(X, method="ward")

dendrogram(linked)
plt.title("Dendrogram")
plt.xlabel("Data Points")
plt.ylabel("Distance")
plt.show()

# Hierarchical clustering
model = AgglomerativeClustering(n_clusters=3, linkage="ward")

labels = model.fit_predict(X)

# Display clusters
data["Cluster"] = labels
print(data)

# Visualize clusters
plt.scatter(X["Age"], X["Marks"], c=labels)
plt.xlabel("Age")
plt.ylabel("Marks")
plt.title("Hierarchical Clustering")
plt.show()
```

Exp 9

Python

```python
import pandas as pd

df = pd.read_csv("./weather.csv")
total = len(df)


yes = len(df[df["PlayTennis"] == "Yes"])
no = len(df[df["PlayTennis"] == "No"])

p_yes = yes / total
p_no = no / total

# test data
values = ["Sunny", "Cool", "High", "Strong"]
attribs = ["Outlook", "Temperature", "Humidity", "Wind"]

for i in range(len(attribs)):
    attrib = attribs[i]
    value = values[i]

    count_yes = len(df[(df[attrib] == value) & (df["PlayTennis"] == "Yes")])
    count_no = len(df[(df[attrib] == value) & (df["PlayTennis"] == "No")])

    p_yes = p_yes * count_yes / yes
    p_no = p_no * count_no / no

print("yes prob", p_yes)
print("no prob", p_no)

if p_yes > p_no:
    print("Play Tennis: Yes")
else:
    print("Play Tennis: No")
```

Exp 10 - page hits

Python

```python
n=3
ite = 10

graph = [
    [0, 1, 1],
    [0, 0, 1],
    [1, 0, 0]
]

auth = [1]*n
hub = [1]*n

for i in range(ite):

    for j in range(n):
        auth[j]=0
        for k in range(n):
            auth[j] += hub[k] * graph[k][j]

    for j in range(n):
        hub[j]=0
        for k in range(n):
            hub[j] += auth[k] * graph[j][k]

    a = sum(auth)
    h = sum(hub)

    auth = [x/a for x in auth]
    hub = [x/h for x in hub]

    print('iteration',i+1)
    print('Auhtority score', auth)
    print('hub score', hub)

print('Final ')
print('Auhtority score', auth)
print('hub score', hub)
```

Exp 10 - page rank

Python

```python
n=3
d=0.85
ite=10

graph = [
    [0, 1, 1],
    [0, 0, 1],
    [1, 0, 0]
]

rank =[1/n, 1/n, 1/n]

for i in range(ite):
    new_rank=[]
    for j in range(n):
        total = 0
        for k in range(n):
            if graph[k][j] == 1:
                total += rank[k] / sum(graph[k])

        pr = (1-d)/n + d*total
        new_rank.append(pr)

    print(new_rank)
    rank=new_rank

print("\n Final rank is", rank)
```